ChatGPT said:

Java Collections and Generics

In Java, **Collections** and **Generics** are two important concepts used to store, manage and process data.

Let's understand them from beginner to interview level with practical examples.

Part 1: Java Collections Framework

1. What is a Collection?

A Collection is an object that stores a group of objects in a single unit.

For example, suppose you want to store five employee names.

Without collections, you might use an array:

```java
String[] employees = {
    "Rahul", "Amit", "Suman", "Ravi", "John"
};
```

Using a collection:

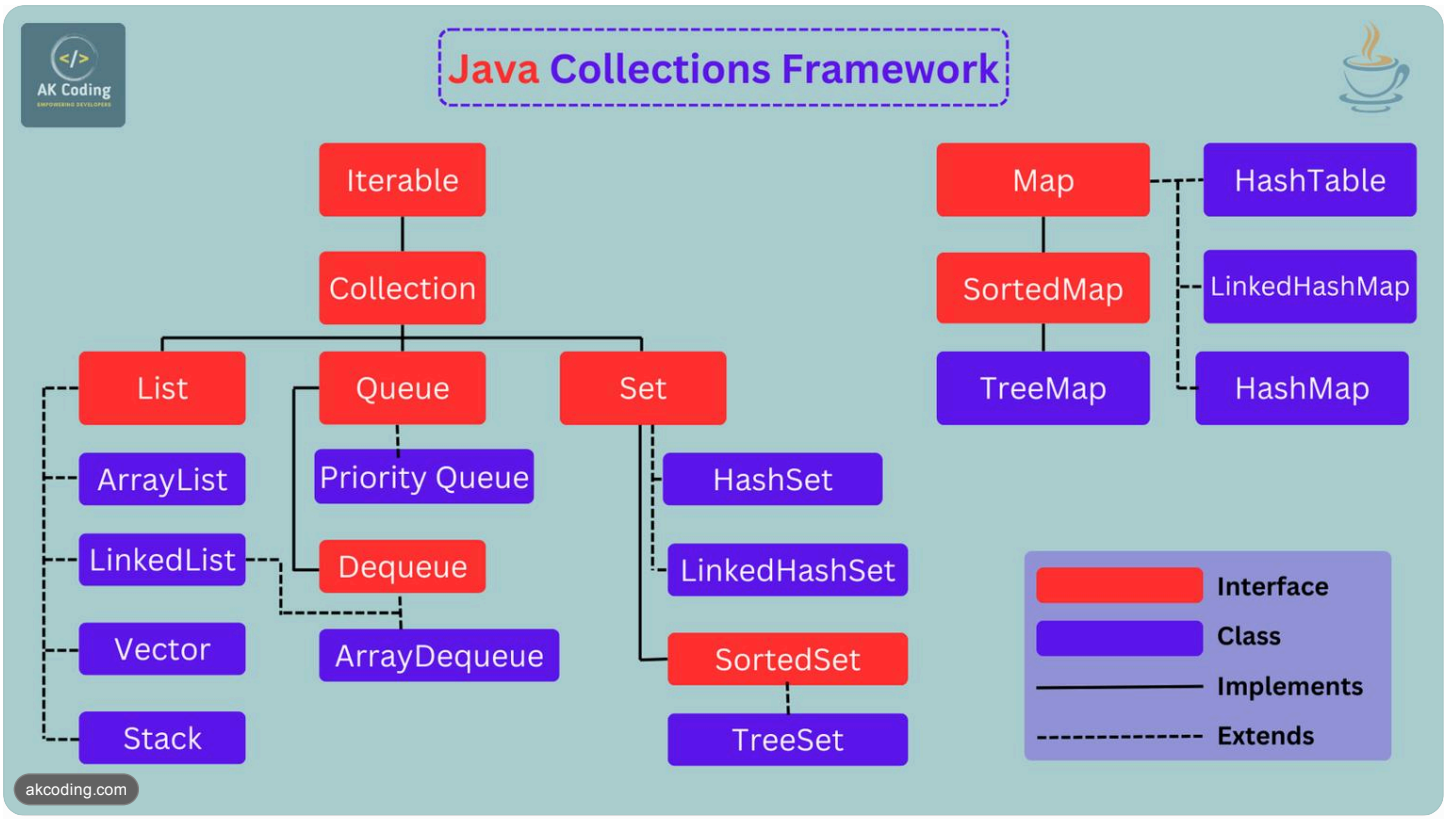
```java
List<String> employees = new ArrayList<>();

employees.add("Rahul");
employees.add("Amit");
employees.add("Suman");
employees.add("Ravi");
employees.add("John");
```

Collections provide flexible ways to add, remove, search, sort and iterate over data.

2. Java Collections Framework hierarchy

The Collections Framework contains interfaces and classes that define different ways to store data.



The main interfaces are:

1. List
2. Set
3. Queue
4. Map (part of the Collections Framework, but not a subtype of Collection)

3. List interface

A List stores elements in an ordered sequence.

Features:

- Allows duplicate values.
- Maintains insertion order.
- Supports index-based access.
- Allows multiple null values in common implementations such as ArrayList.

Example:

Java

```
import java.util.*;

public class Main {
    public static void main(String[] args) {

        List<String> names = new ArrayList<>();

        names.add("Rahul");
        names.add("Amit");
        names.add("Rahul");
        names.add("Suman");

        System.out.println(names);
        System.out.println(names.get(0));
    }
}
```

Output:

Plain text

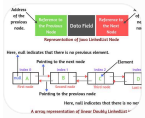
```
[Rahul, Amit, Rahul, Suman]
Rahul
```

Important List implementations



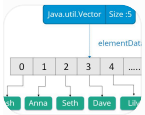
ArrayList

Uses a dynamic array internally. Fast for accessing elements by index, but inserting or deleting in the middle can be expensive.



LinkedList

Uses a doubly linked list. Efficient at adding or removing elements at the ends; accessing an arbitrary index is slower.



Vector

A legacy dynamic-array implementation. Its individual methods are synchronized.

4. Set interface

A Set stores unique elements. It does not allow duplicates.

Features:

- No duplicate elements.
- Does not provide index-based access.
- Ordering depends on the implementation.

Example:

Java

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {

        Set<Integer> numbers = new HashSet<>();

        numbers.add(10);
        numbers.add(20);
        numbers.add(10);
        numbers.add(30);

        System.out.println(numbers);
    }
}
```

Output (order may vary):

Plain text

```
[20, 10, 30]
```

Important Set implementations

Implementation	Ordering	Main purpose
HashSet	No guaranteed order	Fast lookup
LinkedHashSet	Insertion order	Unique values with predictable order

TreeSet	Sorted order	Sorted unique values

5. Queue interface

Queue generally follows FIFO (First In, First Out).

The element inserted first is generally removed first.

Example:

Java

```
import java.util.*;

public class Main {
    public static void main(String[] args) {

        Queue<String> queue = new LinkedList<>();

        queue.add("Rahul");
        queue.add("Amit");
        queue.add("Suman");

        System.out.println(queue.poll());
        System.out.println(queue);
    }
}
```

Output:

Plain text

```
Rahul
[Amit, Suman]
```

Important methods:

Method	Purpose
`add()`	Insert an element
`offer()`	Insert an element
`poll()`	Remove and return the head, or null
`peek()`	View the head without removing it
`remove()`	Remove the head; throws exception if empty

Other implementations include `PriorityQueue` and `ArrayDeque`.

6. Map interface

A Map stores data in key-value pairs.

Example:

Java

```
Map<Integer, String> employees = new HashMap<>();

employees.put(101, "Rahul");
employees.put(102, "Amit");
employees.put(103, "Suman");

System.out.println(employees.get(101));
```

Output:

```
Rahul
```

Features:

- Keys must be unique.
- Values can be duplicated.
- Each key maps to at most one value.
- Ordering depends on implementation.

Important Map implementations

Implementation	Ordering	Main purpose
HashMap	No guaranteed order	General-purpose key-value storage
LinkedHashMap	Insertion order by default	Predictable iteration order
TreeMap	Sorted by key	Sorted key-value data
Hashtable	No guaranteed order	Legacy synchronized map

Remember: `HashMap` allows one null key and multiple null values. `Hashtable` does not allow null keys or values.

Part 2: Java Generics

7. What are Generics?

Generics allow you to write classes, interfaces and methods that work with different data types while maintaining **type safety**.

In simple words, Generics tell Java what type of data you want to store or process.

Without Generics

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {

        List list = new ArrayList();

        list.add("Rahul");
        list.add(100);
        list.add(true);

        String name = (String) list.get(0);

        System.out.println(name);
    }
}
```

Problems:

- Different types of data can be inserted accidentally.
- You need type casting.
- Incorrect casting can cause `ClassCastException`.

With Generics

```java
```

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {

        List<String> list = new ArrayList<>();

        list.add("Rahul");
        list.add("Amit");

        // list.add(100); // Compile-time error

        String name = list.get(0);

        System.out.println(name);
    }
}
```

Benefits:

- Type safety.
- No unnecessary type casting.
- Fewer runtime errors.
- Reusable code.

8. Generic class

You can create your own generic class using a type parameter such as `T`.

Example:

Java

```java
class Box<T> {

    private T value;

    public void setValue(T value) {
        this.value = value;
    }

    public T getValue() {
        return value;
    }
}
```

Using this class:

Java

```java
public class Main {

    public static void main(String[] args) {

        Box<String> box1 = new Box<>();
        box1.setValue("Hello");

        Box<Integer> box2 = new Box<>();
        box2.setValue(100);

        System.out.println(box1.getValue());
        System.out.println(box2.getValue());
    }
}
```

Output:

Plain text

```
Hello
100
```

Here, `T` is a type parameter. It becomes `String` or `Integer` depending on how you create the object.

9. Generic method

A generic method can accept different data types.

Java

```java
public class Main {

    public static <T> void printData(T data) {

        System.out.println(data);
    }

    public static void main(String[] args) {

        printData("Hello");
        printData(100);
        printData(10.50);
    }
}
```

Output:

Plain text

```
Hello
100
10.5
```

Here:

- `<T>` declares a generic type parameter.
- `T data` accepts a value of that type.
- The same method works with different data types.

10. Types of generic parameters

Java commonly uses these naming conventions:

Symbol	Meaning
`T`	Type
`E`	Element
`K`	Key
`V`	Value
`N`	Number

Example:

Java

```java
Map<Integer, String> employees = new HashMap<>();
```

Here:

- `Integer` is the key type.
- `String` is the value type.

11. Bounded generics

Sometimes you want a generic type to accept only a particular category of data.

You can use `extends` to define an upper bound.

Example:

```java
class Calculator<T extends Number> {

    private T value;

    Calculator(T value) {
        this.value = value;
    }

    public double square() {
        return value.doubleValue() * value.doubleValue();
    }
}
```

Usage:

```java
Calculator<Integer> c1 = new Calculator<>(10);
Calculator<Double> c2 = new Calculator<>(5.5);
```

Both work because `Integer` and `Double` extend `Number`.

But this is not allowed:

```java
// Calculator<String> c3 = new Calculator<>("Hello");
```

Because `String` does not extend `Number`.

12. Wildcards in Generics

Java provides `?` as a wildcard, meaning an unknown type.

There are three important forms:

A. Unbounded wildcard: `<?>`

Accepts a collection of any type.

```java
public static void printList(List<?> list) {
    for (Object value : list) {
        System.out.println(value);
    }
}
```

B. Upper bounded wildcard: `<? extends Number>`

Accepts Number or its subclasses.

Java

```
public static double sum(List<? extends Number> numbers) {

    double total = 0;

    for (Number n : numbers) {
        total += n.doubleValue();
    }

    return total;
}
```

C. Lower bounded wildcard: `<? super Integer>`

Accepts Integer or a superclass of Integer.

Java

```
public static void addNumbers(List<? super Integer> numbers) {

    numbers.add(10);
    numbers.add(20);
}
```

A useful rule to remember is **PECS**:

- Producer Extends: use `extends` when you mainly read values.
- Consumer Super: use `super` when you mainly add values.

Part 3: Important Collection methods

These are frequently used in Java 8 and interview coding.

Method	Purpose
`add()`	Add an element
`addAll()`	Add another collection
`remove()`	Remove an element
`contains()`	Check whether an element exists
`size()`	Get the number of elements
`isEmpty()`	Check whether the collection is empty
`clear()`	Remove all elements
`iterator()`	Iterate over elements
`stream()`	Create a Stream
`forEach()`	Process elements

Example:

Java

```
List<Integer> numbers = new ArrayList<>();

numbers.add(10);
numbers.add(20);
numbers.add(30);

System.out.println(numbers.size());       // 3
System.out.println(numbers.contains(20)); // true

numbers.remove(Integer.valueOf(20));
```

```
System.out.println(numbers); // [10, 30]
```

Part 4: Collection vs Collections vs Generics

These three terms are often confusing.

Term	Meaning
`Collection`	Root interface for List, Set and Queue
`Collections`	Utility class containing methods for collection operations
`Generics`	Type-safety mechanism

Example of Collections utility class:

Java

```
List<Integer> numbers =
        new ArrayList<>(Arrays.asList(30, 10, 20));

Collections.sort(numbers);

System.out.println(numbers);
```

Output:

Plain text

```
[10, 20, 30]
```

Other useful methods:

Java

```
Collections.reverse(numbers);
Collections.shuffle(numbers);
Collections.max(numbers);
Collections.min(numbers);
Collections.frequency(numbers, 10);
```

Part 5: Important interview questions

1. What is the difference between ArrayList and LinkedList?
2. What is the difference between HashSet and TreeSet?
3. What is the difference between HashMap and Hashtable?
4. How does HashMap work internally?
5. What happens when duplicate keys are inserted into HashMap?
6. What is the difference between Collection and Collections?
7. Why do we use Generics in Java?
8. What is the difference between `T` and `?`?
9. What is the difference between `<? extends T>` and `<? super T>`?
10. What is type erasure in Java?
11. Can we use primitive types in Generics?
12. What is the difference between Iterator and ListIterator?

Part 6: Test your understanding

Java Collections & Generics Quiz 0/5 answered

1. Which collection allows duplicate elements?

- HashSet
- ArrayList
- TreeSet

2. Which collection stores key-value pairs?

- List
- Set
- Map

3. What is the main benefit of Generics?

- Faster CPU
- Type safety
- Automatic database connection

4. Which implementation stores unique elements in sorted order?

- HashSet
- LinkedHashSet
- TreeSet

5. What does <? extends Number> mean?

- Only String
- Number or its subclasses
- Any Java object

Submit answers

My recommendation: Learn ArrayList, HashMap, HashSet, Stream API and Generics first. These are especially useful for your Spring Boot development and Java interviews.